Life Memory Wall — Technical Documentation

---

1. Technology Stack

Layer	Technology	Version	Purpose
Framework	React	18.3.1	UI component library
Language	TypeScript	~5.6.2	Type safety
Build Tool	Vite	6.0.5	Dev server, bundling, HMR
Styling	Tailwind CSS	3.4.17	Utility-first CSS framework
CSS Processing	PostCSS + Autoprefixer	8.4.x / 10.4.x	CSS transforms and vendor prefixes
Animation	Framer Motion	11.15.0	Declarative animations and transitions
Icons	Lucide React	0.468.0	SVG icon library
State Management	Zustand	5.0.3	Lightweight global state
Routing	React Router DOM	7.1.1	Client-side routing (wrapped but not actively used for route matching)

---

2. Project Structure

```
MemoryWall-Claude/
├── index.html                  # HTML entry point (loads Google Fonts)
├── package.json                # Dependencies and scripts
├── tsconfig.json               # TypeScript configuration
├── tsconfig.app.json           # App-specific TS config
├── tsconfig.node.json          # Node-specific TS config
├── vite.config.ts              # Vite build configuration
├── tailwind.config.js          # Tailwind theme customization
├── postcss.config.js           # PostCSS plugins
├── docs/                       # Documentation (this file)
│
└── src/
    ├── main.tsx                # React entry point (StrictMode + BrowserRouter)
    ├── App.tsx                 # Root component (routing logic)
    ├── index.css               # Global styles + Tailwind directives
    ├── vite-env.d.ts           # Vite type declarations
    │
    ├── types/
    │   └── index.ts            # TypeScript interfaces (7 types)
    │
    ├── store/
    │   └── useStore.ts         # Zustand store (single file, ~256 lines)
```

```
    │
    ├── data/
    │   └── mockData.ts         # Mock users + 13 spaces with memories
    │
    └── components/
        ├── LoginPage.tsx       # Authentication UI (3 login methods)
        ├── SpaceSelector.tsx   # Space grid + create/join/members modals
        ├── Timeline.tsx        # Main timeline view + split-view logic
        ├── MemoryCard.tsx      # Polaroid-style memory card component
        ├── MemoryDetail.tsx    # Memory detail with substory timeline
        ├── CreateMemoryModal.tsx # Memory create/edit form
        ├── FloatingNav.tsx     # Bottom navigation bar
        └── ParticleBackground.tsx # Canvas-based particle animation
```

---

3. Type System

3.1 Core Interfaces (`src/types/index.ts`)

```typescript
interface SubStory {
  id: string
  date: string
  type: 'text' | 'photo' | 'photos'
  title?: string
  content?: string          // for text type
  photos?: string[]         // for photo/photos type
  caption?: string          // for photo/photos type
}

interface Memory {
  id: string
  title: string
  date: string
  endDate?: string          // multi-day events
  photos: string[]
  story: string
  location?: string
  tags?: string[]
  reactions?: Record<string, number>  // emoji -> count
  comments?: Comment[]
  storylineId?: string
  substories?: SubStory[]
  visibleTo?: string[]      // user IDs; empty/undefined = everyone
  createdBy?: string
}

interface Comment {
  id: string
  author: string
  text: string
  date: string
}
```

```typescript
interface JoinRequest {
  userId: string
  userName: string
  requestedAt: string
}

interface SpaceMember {
  userId: string
  name: string
  role: 'owner' | 'admin' | 'member'
  status: 'active' | 'pending'
  joinedAt: string
}

interface MemorySpace {
  id: string
  title: string
  coverImage: string
  coverEmoji: string
  memoryCount: number
  type: 'personal' | 'group'
  createdBy: string
  inviteCode?: string        // group spaces only
  membersList: SpaceMember[]
  joinRequests: JoinRequest[]
  description?: string
  memories: Memory[]
}

interface User {
  id: string
  name: string
  avatar: string
  email: string
  phone?: string
}
```

3.2 Store Interface (`src/store/useStore.ts`)

```typescript
interface AppState {
  // State
  isLoggedIn: boolean
  currentUser: User | null
  spaces: MemorySpace[]
  activeSpaceId: string | null

  // Auth
  login: (user?: Partial<User>) => void
  logout: () => void
```

```
  // Navigation
  setActiveSpace: (id: string | null) => void
  getActiveSpace: () => MemorySpace | undefined
  getVisibleSpaces: () => MemorySpace[]
  getVisibleMemories: (space: MemorySpace) => Memory[]

  // Memory CRUD
  addMemory: (spaceId: string, memory: Memory) => void
  updateMemory: (spaceId: string, memoryId: string, updates: Partial<Memory>) => void
  deleteMemory: (spaceId: string, memoryId: string) => void
  addReaction: (spaceId: string, memoryId: string, emoji: string) => void
  addSubstory: (spaceId: string, memoryId: string, substory: SubStory) => void

  // Space management
  addSpace: (space: MemorySpace) => void
  requestJoinByCode: (code: string) => { success: boolean; spaceName?: string; error?: string }
  approveJoinRequest: (spaceId: string, userId: string) => void
  rejectJoinRequest: (spaceId: string, userId: string) => void
  getPendingRequests: (spaceId: string) => JoinRequest[]
  getInviteCode: (spaceId: string) => string | undefined
  removeMember: (spaceId: string, userId: string) => void
  updateMemberRole: (spaceId: string, userId: string, role: SpaceMember['role']) => void
}
```

---

4. Component Architecture

4.1 Component Tree

```
App
├── LoginPage                    (screen: !isLoggedIn)
│   └── ParticleBackground
│
├── SpaceSelector                (screen: isLoggedIn && !activeSpaceId)
│   └── ParticleBackground
│   └── [Modals: Create / Join / Members]
│
└── Timeline                     (screen: isLoggedIn && activeSpaceId)
    ├── MemoryCard[]             (full mode: all cards)
    ├── [Compact sidebar list]   (split mode: left panel)
    ├── MemoryDetail             (split mode: right panel)
    ├── FloatingNav
    └── CreateMemoryModal
```

4.2 Component Details

`App.tsx` (36 lines)

- Root component, no routing logic — uses conditional rendering
- Reads `isLoggedIn` and `activeSpaceId` from Zustand store
- Three states: LoginPage | SpaceSelector | Timeline
- Wraps Timeline and SpaceSelector in `motion.div` for fade transitions

- Deliberately avoids `AnimatePresence` at root level to prevent stuck transitions

`LoginPage.tsx` (~437 lines)

- **State machine**: `screen` = 'main' | 'email' | 'phone'
- **Email sub-state**: `emailStep` = 'enter' | 'password'
- **Phone sub-state**: `phoneStep` = 'enter' | 'otp'
- OTP implementation: 6 individual controlled inputs with auto-focus chain
- User matching: compares input email/phone against `allUsers` array
- If no match found, creates session with entered credentials but defaults to u1
- Quick login: maps all 6 test users to instant login buttons
- Uses `AnimatePresence mode="wait"` for screen transitions (works here because returns are never null)

`SpaceSelector.tsx` (~418 lines)

- **Modal state**: `modal` = 'none' | 'create' | 'join' | 'members'
- Computes `totalPendingRequests` across all spaces where user is owner/admin
- Space bubbles use staggered spring animations (delay based on index)
- Member modal: sorts members by role priority (owner=0, admin=1, member=2)
- Invite code copy: uses `navigator.clipboard.writeText()`
- Join flow: calls `requestJoinByCode()` and displays result message

`Timeline.tsx` (~326 lines)

- **Key state**: `selectedMemoryId` determines full vs split mode
- Uses `getVisibleMemories(space)` to filter by `visibleTo` field
- Sorts memories chronologically: `new Date(a.date).getTime()` comparison
- **Full mode**: Alternating left/right cards with SVG dashed line
- **Split mode**: 12% sidebar (min-width 180px) + flex-1 detail panel
- Scroll progress tracked via `useScroll()` + `useTransform()` for path animation
- Back button: `goBack()` checks `isDetailOpen` to decide navigation target
- Passes `spaceType`, `members`, `currentUserId` to CreateMemoryModal

`MemoryCard.tsx` (~163 lines)

- Receives `onCardClick` prop — click handler with `e.stopPropagation()`
- Polaroid styling via `.polaroid` CSS class (custom padding + shadow)
- Slight rotation: `style={{ rotate: side === 'left' ? -1.5 : 1.5 }}`
- Reaction picker: toggleable emoji palette that appears above card
- Edit/Delete buttons: visible on hover via group-hover opacity transition
- Tag-to-emoji mapping for photo area placeholder icons
- Substory hint: shows count + "tap to explore" message

`MemoryDetail.tsx` (~381 lines)

- Two tabs managed by `activeTab` state: 'timeline' | 'photos'
- Substory grouping: `groupedByDate` object built by iterating substories
- Dates sorted chronologically for display order
- "Add a moment" form: collapsible, creates SubStory and calls `onAddSubstory`
- Photos tab: extracts all photo/photos type substories into flat array
- No enclosing card — content renders directly on gradient background
- Sticky positioning: `sticky top-24 h-[calc(100vh-7rem)]` with `overflow-hidden`

`CreateMemoryModal.tsx` (~240 lines)

- Dual mode: create (empty form) vs edit (pre-filled from `editMemory`)
- `useEffect` resets/fills form fields when modal opens or editMemory changes

- Visibility selector: only renders for group spaces with >1 member
- `visibleTo` state: array of user IDs, toggled via checkboxes
- If visibility enabled but no members selected, `visibleTo` is omitted (everyone can see)
- Memory ID generation: `m-${Date.now()}` for new memories

`FloatingNav.tsx` (~62 lines)

- Fixed position: `fixed bottom-6 left-1/2 -translate-x-1/2 z-40`
- Glass morphism container with rounded-full
- Create button: gold-to-coral gradient with blur shadow and glow animation
- Spring animation on mount: delayed entry from bottom

`ParticleBackground.tsx` (~81 lines)

- Canvas-based particle system using `requestAnimationFrame`
- 50 particles with random position, size (1-4px), speed, opacity, and hue
- Two color families: gold `rgba(212,165,116)` and lavender `rgba(200,180,230)`
- Particles wrap around screen edges (toroidal topology)
- Handles window resize events
- Cleanup: cancels animation frame and removes resize listener on unmount
- Renders as `fixed inset-0 pointer-events-none z-0`

---

5. State Management Details

5.1 Zustand Store Architecture

Single store pattern — all state in one `create<AppState>()` call. No middleware, no persistence, no devtools.

State updates use immutable patterns:

```
// Nested update example (updating a memory inside a space):
set((state) => ({
  spaces: state.spaces.map((s) =>
    s.id === spaceId
      ? { ...s, memories: s.memories.map((m) =>
          m.id === memoryId ? { ...m, ...updates } : m
        )}
      : s
  ),
}))
```

5.2 Key Store Functions

`generateUniqueCode(existingCodes)` — Internal utility:

- Character set: `ABCDEFGHJKLMNPQRSTUVWXYZ23456789` (excludes confusing chars: 0, O, 1, I, L)
- Generates 6-character codes
- Uses Set for O(1) collision checking
- Do-while loop ensures uniqueness

`login(user?)`:

- Accepts optional `Partial<User>`
- Merges with existing user from `allUsers` array (matched by `user.id`)

- Falls back to allUsers[0] if no match
- Constructs complete User object with all required fields

`getVisibleSpaces()` :

- Filters `spaces` where `currentUser.id` exists in `membersList` with `status: 'active'`
- Returns empty array if not logged in

`getVisibleMemories(space)` :

- If `memory.visibleTo` is empty/undefined -> visible to all
- If `memory.visibleTo` has entries -> only visible if `currentUser.id` is in the array

`requestJoinByCode(code)` :

- Uppercases and trims input
- Validates against 3 conditions: code exists, not already member, no pending request
- Returns `{ success, spaceName?, error? }` for UI feedback

`addSpace(space)` :

- Auto-generates invite code for group spaces
- Adds creator as owner in `membersList`
- Initializes empty `joinRequests`

5.3 State Flow Diagram

```
User Action -> Component Handler -> Store Method -> set() -> React Re-render

Example: Adding a memory
1. User fills form in CreateMemoryModal
2. handleSave() in Timeline creates Memory object
3. Calls addMemory(spaceId, memory) on store
4. Store runs set() with new spaces array (memory appended)
5. Timeline component re-reads space.memories
6. getVisibleMemories() filters the list
7. Sorted memories re-render on screen
```

---

6. Styling Architecture

6.1 Tailwind Configuration (`tailwind.config.js`)

Custom Colors:

```
colors: {
  lavender: '#f0e6ff',   // Light purple — backgrounds, accents
  peach: '#ffe8d6',       // Warm peach — backgrounds, gradients
  gold: '#d4a574',        // Amber gold — primary accent, CTAs
  coral: '#e8927c',       // Soft red — secondary accent, alerts
  teal: '#7cb5a8',        // Sage green — success, admin
  warmWhite: '#fffbf5',   // Off-white — card backgrounds
  warmDark: '#4a3728',    // Dark brown — primary text
  warmMid: '#8b7355',     // Medium brown — secondary text (mostly replaced with warmDark/opacity)
}
```

Custom Fonts:

```
fontFamily: {
  serif: ['"Playfair Display"', 'Georgia', 'serif'],
  sans: ['"DM Sans"', 'system-ui', 'sans-serif'],
  handwriting: ['"Caveat"', 'cursive'],
}
```

Custom Animations:

Name	Duration	Behavior
float	6s	Gentle up/down bob (translateY 0 -> -20px -> 0)
float-slow	8s	Slower version
float-slower	10s	Slowest version
glow	2s	Pulsing box-shadow (alternate)
gradient	15s	Background position animation
fade-up	0.8s	Opacity 0->1 + translateY 30->0
particle	20s	Full-screen diagonal drift with rotation

6.2 Global CSS (`src/index.css`)

Key custom classes:

```
.gradient-bg     /* Animated 4-color gradient background (lavender/peach/blue/warm) */
.glass           /* Glassmorphism: semi-transparent white + 20px blur + white border */
.glass-dark      /* Dark glassmorphism variant (subtle) */
.text-shadow-warm /* Gold-tinted text shadow */
.polaroid        /* Card style: off-white bg, thick bottom padding, warm shadow */
.timeline-path   /* Animated dashed stroke for SVG timeline line */
```

Scrollbar styling: Custom thin scrollbar with gold-tinted thumb.

6.3 Text Color Strategy

All user-facing text uses `text-warmDark` (the darkest brown) with **Tailwind opacity modifiers** to create a hierarchy:

Opacity	Usage	Example
100% (no modifier)	Primary headings, titles	`text-warmDark`
/70	Important secondary text	`text-warmDark/70`
/65	Body text, story content	`text-warmDark/65`
/60	Labels, descriptions	`text-warmDark/60`
/55	Captions, dates, subtitles	`text-warmDark/55`

/50	Metadata, member info	`text-warmDark/50`
/45	Hints, placeholders	`text-warmDark/45`
/40	De-emphasized text	`text-warmDark/40`
/35	Very subtle text, empty states	`text-warmDark/35`

This ensures all text is readable against the light gradient background. Earlier versions used `text-warmMid` (medium brown) which was too light.

---

7. Animation System

7.1 Framer Motion Usage

Page transitions (App.tsx):

```
<motion.div
  key={`space-${activeSpaceId}`}
  initial={{ opacity: 0 }}
  animate={{ opacity: 1, transition: { duration: 0.4 } }}
/>
```

Card entrance (MemoryCard.tsx):

```
initial={{ opacity: 0, x: side === 'left' ? -60 : 60, y: 20 }}
whileInView={{ opacity: 1, x: 0, y: 0 }}
viewport={{ once: true, margin: '-50px' }}
transition={{ duration: 0.8, delay: 0.1, ease: 'easeOut' }}
```

Spring physics (various):

```
transition={{ type: 'spring', stiffness: 200, damping: 25 }}
```

Scroll-linked animation (Timeline.tsx):

```
const { scrollYProgress } = useScroll()
const pathLength = useTransform(scrollYProgress, [0, 1], [0, 1])
// Used for timeline path reveal + mobile line scale
```

Staggered animations (SpaceSelector bubbles):

```
transition={{ duration: 0.6, delay: i * 0.1, type: 'spring', stiffness: 100 }}
```

7.2 CSS Animations

```
@keyframes gradient { /* Background color shift — 15s infinite */ }
@keyframes dash { /* Timeline stroke animation — 30s linear infinite */ }
```

```
/* Plus Tailwind keyframes: float, glow, fadeUp, particle */
```

---

8. Canvas Rendering (ParticleBackground)

```
Setup:
  - Canvas sized to window (responsive)
  - 50 particles initialized with random properties
  - Two color families: gold and lavender

Render loop (requestAnimationFrame):
  1. Clear canvas
  2. For each particle:
     a. Update position: x += speedX, y += speedY
     b. Wrap edges (toroidal)
     c. Draw filled circle at (x, y) with size and opacity
  3. Request next frame

Performance considerations:
  - pointer-events: none (no interaction overhead)
  - z-index: 0 (behind all content)
  - Cleanup on unmount (cancelAnimationFrame + remove resize listener)
  - 50 particles is lightweight (~50 arc draws per frame)
```

---

9. Responsive Design Implementation

9.1 Breakpoint Strategy

Primary breakpoint: `md` (768px) — Tailwind's medium breakpoint.

Feature	Mobile (<768px)	Desktop (>=768px)
Timeline cards	Single column, left-aligned	Alternating left/right
Timeline dots	Left side (18px)	Center (50%)
Memory detail	Full-screen overlay	Side panel (flex-1)
Split sidebar	Hidden	12% width, min 180px
Space bubbles	112x112px	144x144px
Title sizes	text-3xl	text-5xl

9.2 Key Responsive Patterns

```
/* Hidden on mobile, shown on desktop */
className="hidden md:block"

/* Full-screen mobile overlay */
className="fixed inset-0 z-40 md:hidden"
```

```
/* Responsive width */
className="w-28 h-28 md:w-36 md:h-36"

/* Conditional layout */
className={`${isDetailOpen ? 'w-[12%] min-w-[180px] hidden md:block' : 'w-full'}`}
```

---

10. Build & Development

10.1 Scripts

```
npm run dev      # Start Vite dev server (HMR enabled)
npm run build    # TypeScript check + Vite production build
npm run preview  # Preview production build locally
```

10.2 Dev Server

```
npx vite --host 0.0.0.0  # Bind to all interfaces (needed for some setups)
# Default: http://localhost:5173
```

10.3 Google Fonts Loading

Fonts are loaded via `<link>` tags in `index.html`:

- Playfair Display: weights 400-700
- DM Sans: weights 400-500-600
- Caveat: weights 400-700

10.4 Important Development Notes

1. **No backend**: All data is in-memory via Zustand. Refreshing the page resets to mock data.
2. **HMR state persistence**: Zustand state survives HMR updates. If `isLoggedIn` is true from a previous session, you may not see the login page. Use hard refresh (Cmd+Shift+R) to reset.
3. **No actual auth**: Login methods don't validate credentials. Any input is accepted.
4. **No actual file upload**: Photo upload areas are placeholders showing "Coming soon".
5. **No actual OTP**: Any 6-digit code is accepted for phone login.
6. **Tailwind dynamic classes**: Avoid `space-y-${variable}` patterns — Tailwind purges them at build time. Always use full class names.

---

11. File Size Summary

File	Lines	Description
`types/index.ts`	71	All TypeScript interfaces
`store/useStore.ts`	256	Zustand store with all actions
`data/mockData.ts`	332	6 users + 13 spaces with memories
`App.tsx`	36	Root conditional rendering
`main.tsx`	14	React DOM mount

`index.css`	90	Global styles + Tailwind
`LoginPage.tsx`	~437	Authentication (3 methods)
`SpaceSelector.tsx`	~418	Space grid + 3 modals
`Timeline.tsx`	~326	Timeline + split view
`MemoryCard.tsx`	~163	Polaroid card component
`MemoryDetail.tsx`	~381	Detail view + substories
`CreateMemoryModal.tsx`	~240	Create/edit memory form
`FloatingNav.tsx`	~62	Bottom nav bar
`ParticleBackground.tsx`	~81	Canvas particles
Total	**~2,907**	**Full application**

---

12. Known Limitations & Future Work

Current Limitations

1. **No persistence** — data resets on page refresh (no localStorage, no backend)
2. **No real authentication** — no Firebase, no OAuth, no JWT
3. **No image upload** — photo areas are gradient placeholders
4. **No real-time updates** — single-user session, no WebSocket/SSE
5. **No search** — no memory search or filtering beyond visibility
6. **Comment system** — typed but not implemented in UI
7. **Member removal/role change** — store methods exist but no UI buttons

Potential Enhancements

1. **Backend**: Supabase or Firebase for auth + Firestore/Postgres for data
2. **Image hosting**: Cloudinary or S3 for photo uploads
3. **Real-time**: Firestore listeners or WebSocket for live group updates
4. **Search & filter**: By date range, tags, location, member
5. **Notifications**: Push notifications for join requests, new memories
6. **Export**: PDF/image export of timelines
7. **Themes**: Dark mode, custom color themes per space
8. **Map view**: Plot memories on a map by location
9. **Comments**: Threaded discussion on memories
10. **Admin panel**: Full member management with role changes and removal

---

Document generated for Life Memory Wall v1.0.0 Last updated: March 2026